

Modelos de aprendizaje supervisado - Regresión

Ruete, D. (2026). *Modelos de aprendizaje supervisado - Regresión*.
[Apunte]. Universidad Andrés Bello, Santiago, Chile.

Tabla de contenidos

1. Contexto del dataset y preparación del entorno.....	3
1.1. Variables del dataset.....	3
1.2. Instalación de paquetes.....	3
1.3. Bibliotecas utilizadas.....	4
1.4. Carga inicial del dataset.....	5
1.4.1. Resultado esperado e interpretación.....	6
2. Preparación de datos para modelos de Machine Learning.....	7
2.1. Importación de herramientas para preparación de datos.....	7
2.2. Separación entre características y variable objetivo.....	7
2.3. División en entrenamiento y prueba.....	8
2.4. Escalamiento de características.....	9
2.4.1. Interpretación del resultado escalado.....	10
3. Regresión lineal.....	11
3.1. Implementación en Python.....	11
3.2. Resultados obtenidos.....	12
4. Árbol de decisión para regresión.....	14
4.1. Implementación en Python.....	14
4.2. Resultados obtenidos.....	15
4.2.1. Interpretación.....	15
5. Red neuronal MLP para regresión.....	16
5.1. Implementación en Python.....	16
5.2. Resultados obtenidos.....	17
5.2.1 Interpretación.....	17
6. Comparación e interpretación integrada de modelos.....	18
6.1. Código de comparación.....	18
6.2. Tabla comparativa de resultados.....	19
6.3. Visualización de resultados.....	20
6.3.1. Interpretación integrada.....	21
Recursos pedagógicos de la unidad.....	22
Síntesis final.....	24

1. Contexto del dataset y preparación del entorno

Este apunte de clases desarrolla, paso a paso, un cuaderno Colab orientado a la implementación de modelos supervisados de regresión. El propósito es que el estudiante comprenda no solo qué código se ejecuta, sino también por qué se ejecuta, qué resultado produce y cómo se interpreta en el contexto de un problema predictivo.

El caso utilizado corresponde al dataset California Housing. Este conjunto de datos contiene información demográfica y habitacional de distritos censales de California en 1990. La tarea de Machine Learning consiste en predecir el valor mediano de las viviendas a partir de variables de entrada asociadas a ingreso, antigüedad de las casas, composición del hogar, población y ubicación geográfica.

1.1. Variables del dataset

Variable	Descripción
MedInc	Ingreso mediano del distrito, expresado en decenas de miles de dólares. Suele ser una variable altamente informativa para explicar precios de vivienda.
HouseAge	Edad media de las casas del distrito. Permite incorporar antigüedad de las viviendas como característica predictiva.
AveRooms	Número medio de habitaciones por vivienda. Resume el tamaño promedio de las viviendas del distrito.
AveBedrms	Número medio de dormitorios por vivienda. Ayuda a caracterizar la distribución interna de las viviendas.
Population	Población del distrito. Entrega una medida del tamaño demográfico de la zona.
AveOccup	Número medio de ocupantes por hogar. Puede asociarse a densidad habitacional.
Latitude	Latitud del distrito. Representa ubicación geográfica norte-sur.
Longitude	Longitud del distrito. Representa ubicación geográfica este-oeste.
MedHouseVal	Variable objetivo. Valor mediano de las casas del distrito, expresado en cientos de miles de dólares.

1.2. Instalación de paquetes

En Google Colab, muchas bibliotecas científicas ya vienen instaladas. Sin embargo, para que el código sea reproducible en un entorno local, es recomendable incluir explícitamente la instalación de paquetes. En una celda de Colab se utiliza el signo de exclamación para ejecutar instrucciones de sistema.

```
# Instalación de paquetes necesarios en Google Colab o Jupyter
!pip install pandas numpy scikit-learn matplotlib seaborn
```

Línea de código	Explicación
!pip install pandas numpy scikit-learn matplotlib seaborn	Instala las bibliotecas necesarias para manipulación de datos, cálculo numérico, modelos de Machine Learning y visualización. El signo ! permite ejecutar el comando desde una celda de notebook.

1.3. Bibliotecas utilizadas

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.datasets import fetch_california_housing
```

Línea de código	Explicación
import pandas as pd	Importa pandas con el alias pd. Pandas permite trabajar con DataFrames, que son tablas de datos con filas y columnas.
import numpy as np	Importa NumPy con el alias np. NumPy permite realizar operaciones numéricas eficientes con arreglos y matrices.
import matplotlib.pyplot as plt	Importa pyplot, módulo de matplotlib usado para construir gráficos básicos.
import seaborn as sns	Importa seaborn, biblioteca que facilita gráficos estadísticos más elaborados.
from sklearn.datasets import fetch_california_housing	Importa la función que carga el dataset California Housing desde scikit-learn. En Colab normalmente se descarga de forma automática.

1.4. Carga inicial del dataset

```
# Cargar el dataset de California Housing

housing = fetch_california_housing(as_frame=True)


# Crear un DataFrame de pandas

df = housing.frame


# Mostrar las primeras 5 filas del DataFrame

print("Primeras 5 filas del dataset:")

display(df.head())


# Verificar si hay valores nulos en el dataset

print("

Recuento de valores nulos por columna:")

display(df.isnull().sum())
```

Línea de código	Explicación
housing = fetch_california_housing(as_frame=True)	Carga el dataset. El argumento as_frame=True solicita que los datos se entreguen en formato DataFrame de pandas, lo que facilita su análisis tabular.
df = housing.frame	Extrae desde el objeto housing la tabla completa, incluyendo variables predictoras y variable objetivo.
print("Primeras 5 filas del dataset:")	Muestra un mensaje descriptivo antes de presentar los datos.
display(df.head())	Muestra las primeras cinco observaciones. Esta inspección inicial permite verificar nombres de columnas, tipos de variables y valores esperados.
print("\nRecuento de valores nulos por columna:")	Imprime un salto de línea y un título para la revisión de datos faltantes.
display(df.isnull().sum())	Calcula cuántos valores nulos existen en cada columna. En este dataset, el resultado esperado es 0 en todas las variables.

1.4.1. Resultado esperado e interpretación

Las primeras cinco filas permiten observar que cada registro representa un distrito censal. Las columnas MedInc, HouseAge, AveRooms, AveBedrms, Population, AveOccup, Latitude y Longitude actúan como variables de entrada. La columna MedHouseVal corresponde a la variable objetivo que se desea predecir.

Columna	Valores nulos
MedInc	0
HouseAge	0
AveRooms	0
AveBedrms	0
Population	0
AveOccup	0
Latitude	0
Longitude	0
MedHouseVal	0

La ausencia de valores nulos simplifica el proceso inicial, ya que no se requiere imputación, eliminación de registros ni tratamiento especial de datos faltantes. Sin embargo, en proyectos reales esta verificación debe realizarse siempre antes de entrenar un modelo.

2. Preparación de datos para modelos de Machine Learning

Antes de entrenar modelos supervisados es necesario preparar los datos. Esta preparación incluye separar las variables predictoras de la variable objetivo, dividir los datos en entrenamiento y prueba, y escalar las características cuando el algoritmo puede verse afectado por diferencias de escala.

2.1. Importación de herramientas para preparación de datos

```
from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler
```

Línea de código	Explicación
<code>from sklearn.model_selection import train_test_split</code>	Importa la función que permite dividir el dataset en subconjuntos de entrenamiento y prueba.
<code>from sklearn.preprocessing import StandardScaler</code>	Importa el escalador estándar, que transforma cada característica para que tenga media 0 y desviación estándar 1.

2.2. Separación entre características y variable objetivo

```
# 1. Separar características (X) y variable objetivo (y)

X = df.drop('MedHouseVal', axis=1)

y = df['MedHouseVal']

print(f"Dimensiones de X (características): {X.shape}")

print(f"Dimensiones de y (variable objetivo): {y.shape}")
```

Línea de código	Explicación
<code>X = df.drop('MedHouseVal', axis=1)</code>	Crea la matriz de características X eliminando la columna MedHouseVal. El argumento axis=1 indica que se elimina una columna.
<code>y = df['MedHouseVal']</code>	Crea el vector objetivo y seleccionando solamente la columna que se desea predecir.
<code>print(f"Dimensiones de X (características): {X.shape}")</code>	Imprime el tamaño de X. El resultado indica número de observaciones y número de variables de entrada.
<code>print(f"Dimensiones de y (variable objetivo): {y.shape}")</code>	Imprime el tamaño de y. Debe tener la misma cantidad de observaciones que X.

Objeto	Dimensión obtenida	Interpretación
X	(20640, 8)	Existen 20.640 registros y 8 variables de entrada.
y	(20640,)	Existen 20.640 valores objetivo, uno por cada registro.

La coincidencia en el número de filas entre X e y es fundamental. Si X tiene más o menos registros que y, el entrenamiento del modelo no puede realizarse correctamente porque cada fila de entrada debe tener su correspondiente valor objetivo.

2.3. División en entrenamiento y prueba

2. Dividir el dataset en conjuntos de entrenamiento y prueba

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
```

```
print(f"Dimensiones de X_train: {X_train.shape}")
print(f"Dimensiones de X_test: {X_test.shape}")
print(f"Dimensiones de y_train: {y_train.shape}")
print(f"Dimensiones de y_test: {y_test.shape}")
```

Línea de código	Explicación
<code>train_test_split(X, y, test_size=0.20, random_state=42)</code>	Divide simultáneamente X e y. El 20% se reserva para prueba y el 80% para entrenamiento.
<code>X_train</code>	Contiene las variables de entrada utilizadas para entrenar los modelos.
<code>X_test</code>	Contiene las variables de entrada utilizadas para evaluar modelos con datos no vistos durante el entrenamiento.
<code>y_train</code>	Contiene los valores reales asociados al conjunto de entrenamiento.
<code>y_test</code>	Contiene los valores reales asociados al conjunto de prueba.
<code>random_state=42</code>	Fija una semilla aleatoria para que la partición sea reproducible. Esto permite que distintos estudiantes obtengan la misma división y los mismos resultados.

Objeto	Dimensión obtenida	Interpretación
X_train	(16512, 8)	80% de las observaciones y 8 variables predictoras.
X_test	(4128, 8)	20% de las observaciones reservadas para evaluación.
y_train	(16512,)	Valores reales para entrenar.
y_test	(4128,)	Valores reales para comparar con las predicciones.

Esta división evita evaluar el modelo sobre los mismos datos usados para entrenarlo. Si se evaluara solo con datos de entrenamiento, la medición podría ser demasiado optimista y no reflejar la capacidad real de generalización.

2.4. Escalamiento de características

```
# 3. Escalar las características

scaler = StandardScaler()

scaler.fit(X_train)

X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("
Primeras 5 filas de X_train (escalado):
")

display(pd.DataFrame(X_train_scaled, columns=X.columns).head())
```

Línea de código	Explicación
scaler = StandardScaler()	Crea un objeto escalador. Este objeto calculará media y desviación estándar para cada variable.
scaler.fit(X_train)	Ajusta el escalador usando solo los datos de entrenamiento. Esto es importante para evitar fuga de información desde el conjunto de prueba.
X_train_scaled = scaler.transform(X_train)	Transforma el conjunto de entrenamiento usando las medias y desviaciones calculadas.

<code>X_test_scaled = scaler.transform(X_test)</code>	Transforma el conjunto de prueba usando exactamente los mismos parámetros aprendidos desde el entrenamiento.
<code>pd.DataFrame(X_train_scaled, columns=X.columns).head()</code>	Convierte la matriz escalada en DataFrame para visualizarla con los mismos nombres de columnas.

2.4.1. Interpretación del resultado escalado

Después del escalamiento, las variables dejan de estar en sus unidades originales. Por ejemplo, Population ya no expresa cantidad de habitantes, sino cuántas desviaciones estándar se encuentra un valor respecto del promedio del conjunto de entrenamiento. Esta transformación favorece especialmente a modelos sensibles a escala, como las redes neuronales.

3. Regresión lineal

Concepto

La regresión lineal es un modelo supervisado utilizado para predecir una variable numérica continua. Su supuesto principal es que existe una relación aproximadamente lineal entre las variables de entrada y la variable objetivo. En problemas con múltiples variables, el modelo ajusta un hiperplano que minimiza los errores cuadrados entre valores reales y valores predichos.

La ecuación general puede expresarse como: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$. En esta expresión, β_0 representa el intercepto, β_i representa el peso o coeficiente asociado a cada característica, y ϵ representa el error no explicado por el modelo.

3.1. Implementación en Python

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# 1. Inicializar el modelo de Regresión Lineal
linear_model = LinearRegression()

# 2. Entrenar el modelo
linear_model.fit(X_train_scaled, y_train)
print("Modelo de Regresión Lineal entrenado.")

# 3. Realizar predicciones en el conjunto de prueba
y_pred_linear = linear_model.predict(X_test_scaled)
print("
Primeras 5 predicciones del modelo de Regresión Lineal:")
display(y_pred_linear[:5])

# 4. Evaluar el modelo
mse_linear = mean_squared_error(y_test, y_pred_linear)
r2_linear = r2_score(y_test, y_pred_linear)

print(f"
--- Evaluación del Modelo de Regresión Lineal ---")
print(f"Error Cuadrático Medio (MSE): {mse_linear:.4f}")
print(f"Coeficiente de Determinación (R^2): {r2_linear:.4f}")
```

Línea de código	Explicación
<code>from sklearn.linear_model import LinearRegression</code>	Importa el algoritmo de regresión lineal desde scikit-learn.
<code>from sklearn.metrics import mean_squared_error, r2_score</code>	Importa las métricas MSE y R^2 para evaluar el desempeño predictivo.
<code>import numpy as np</code>	Importa NumPy. En este bloque puede no ser indispensable, pero suele utilizarse en operaciones numéricas complementarias.
<code>linear_model = LinearRegression()</code>	Crea una instancia del modelo. En este punto el modelo todavía no ha aprendido de los datos.
<code>linear_model.fit(X_train_scaled, y_train)</code>	Entrena el modelo usando las variables escaladas y los valores reales del conjunto de entrenamiento.
<code>y_pred_linear = linear_model.predict(X_test_scaled)</code>	Genera predicciones para los datos de prueba. Estas predicciones se compararán con <code>y_test</code> .
<code>display(y_pred_linear[:5])</code>	Muestra las primeras cinco predicciones para revisar el formato de salida del modelo.
<code>mse_linear = mean_squared_error(y_test, y_pred_linear)</code>	Calcula el Error Cuadrático Medio comparando valores reales y predichos.
<code>r2_linear = r2_score(y_test, y_pred_linear)</code>	Calcula el coeficiente de determinación R^2 .
<code>print(f"Error Cuadrático Medio (MSE): {mse_linear:.4f}")</code>	Imprime el MSE con cuatro decimales.
<code>print(f"Coeficiente de Determinación (R^2): {r2_linear:.4f}")</code>	Imprime el R^2 con cuatro decimales.

3.2. Resultados obtenidos

Resultado	Valor
Primeras 5 predicciones	[0.7191, 1.7640, 2.7097, 2.8389, 2.6047]
MSE	0.5559
R^2	0.5758

3.2.1. Interpretación

El MSE de 0.5559 indica el promedio de los errores al cuadrado. Como MedHouseVal está expresada en cientos de miles de dólares, la raíz cuadrada del MSE es aproximadamente 0.75, lo que representa un error típico cercano a 75.000 dólares. Esta interpretación debe tratarse como una aproximación didáctica, porque RMSE es la métrica que vuelve explícitamente a la escala original.

El R^2 de 0.5758 indica que la regresión lineal explica aproximadamente el 57.58% de la variabilidad observada en el valor mediano de las viviendas. El porcentaje restante puede deberse a relaciones no lineales, variables omitidas, ruido de los datos o limitaciones propias del modelo lineal.

La principal ventaja de la regresión lineal es su interpretabilidad: cada coeficiente puede asociarse con el efecto estimado de una variable, manteniendo las demás constantes. Su limitación es que puede subestimar relaciones complejas cuando el fenómeno no es lineal.

4. Árbol de decisión para regresión

Concepto

Un árbol de decisión para regresión divide el espacio de características en regiones más pequeñas. En cada región, la predicción corresponde normalmente al promedio de la variable objetivo de las observaciones que llegan a esa hoja. A diferencia de la regresión lineal, este modelo puede capturar relaciones no lineales e interacciones entre variables.

4.1. Implementación en Python

```
from sklearn.tree import DecisionTreeRegressor

from sklearn.metrics import mean_squared_error, r2_score


# 1. Inicializar el modelo de Árbol de Decisión para Regresión
decision_tree_model = DecisionTreeRegressor(max_depth=5,
random_state=42)


# 2. Entrenar el modelo
decision_tree_model.fit(X_train_scaled, y_train)
print("Modelo de Árbol de Decisión entrenado.")


# 3. Realizar predicciones en el conjunto de prueba
y_pred_tree = decision_tree_model.predict(X_test_scaled)
print("
Primeras 5 predicciones del modelo de Árbol de Decisión:")
display(y_pred_tree[:5])


# 4. Evaluar el modelo
mse_tree = mean_squared_error(y_test, y_pred_tree)
r2_tree = r2_score(y_test, y_pred_tree)


print(f"
--- Evaluación del Modelo de Árbol de Decisión ---")
print(f"Error Cuadrático Medio (MSE): {mse_tree:.4f}")
print(f"Coeficiente de Determinación (R^2): {r2_tree:.4f}")
```

Línea de código	Explicación
<code>from sklearn.tree import DecisionTreeRegressor</code>	Importa el modelo de árbol de decisión para problemas de regresión.
<code>decision_tree_model = DecisionTreeRegressor(max_depth=5, random_state=42)</code>	Crea el modelo. <code>max_depth=5</code> limita la profundidad del árbol para reducir riesgo de sobreajuste. <code>random_state</code> asegura reproducibilidad.
<code>decision_tree_model.fit(X_train_scaled, y_train)</code>	Entrena el árbol usando los datos de entrenamiento. Aunque los árboles no requieren necesariamente escalamiento, aquí se usa la misma versión escalada para mantener consistencia comparativa.
<code>y_pred_tree = decision_tree_model.predict(X_test_scaled)</code>	Genera predicciones sobre el conjunto de prueba.
<code>display(y_pred_tree[:5])</code>	Muestra las primeras cinco predicciones del árbol.
<code>mse_tree = mean_squared_error(y_test, y_pred_tree)</code>	Calcula el MSE del árbol de decisión.
<code>r2_tree = r2_score(y_test, y_pred_tree)</code>	Calcula el R^2 del árbol de decisión.
<code>print(...)</code>	Imprime las métricas con formato de cuatro decimales.

4.2. Resultados obtenidos

Resultado	Valor
Primeras 5 predicciones	[1.1686, 1.2716, 3.1996, 2.2048, 1.6428]
MSE	0.5245
R^2	0.5997

4.2.1. Interpretación

El árbol de decisión obtiene un MSE menor que la regresión lineal y un R^2 mayor. Esto sugiere que el árbol captura mejor ciertas relaciones no lineales presentes en los datos. Sin embargo, su mejora es moderada, probablemente porque la profundidad máxima se limitó a 5 para controlar el sobreajuste.

El parámetro `max_depth` es clave. Si se permite que el árbol crezca sin control, puede memorizar patrones específicos del conjunto de entrenamiento y perder capacidad de generalización. Por esta razón, los árboles requieren especial atención al compromiso entre sesgo y varianza.

5. Red neuronal MLP para regresión

Concepto

El Perceptrón Multicapa o MLP es una red neuronal artificial formada por capas de neuronas interconectadas. Para regresión, el modelo aprende una función que aproxima una salida continua. Su capacidad principal es capturar relaciones no lineales complejas, aunque generalmente sacrifica interpretabilidad frente a modelos como la regresión lineal.

5.1. Implementación en Python

```
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error, r2_score

# 1. Inicializar el modelo de Red Neuronal
mlp_model = MLPRegressor(
    hidden_layer_sizes=(100,),
    activation='relu',
    solver='adam',
    max_iter=500,
    random_state=42,
    early_stopping=True
)

# 2. Entrenar el modelo
mlp_model.fit(X_train_scaled, y_train)
print("Modelo de Red Neuronal (MLP) entrenado.")

# 3. Realizar predicciones en el conjunto de prueba
y_pred_mlp = mlp_model.predict(X_test_scaled)
print("Primeras 5 predicciones del modelo de Red Neuronal:")
display(y_pred_mlp[:5])

# 4. Evaluar el modelo
mse_mlp = mean_squared_error(y_test, y_pred_mlp)
r2_mlp = r2_score(y_test, y_pred_mlp)
print(f"--- Evaluación del Modelo de Red Neuronal (MLP) ---")
print(f"Error Cuadrático Medio (MSE): {mse_mlp:.4f}")
print(f"Coeficiente de Determinación (R^2): {r2_mlp:.4f}")
```

Línea de código	Explicación
<code>from sklearn.neural_network import MLPRegressor</code>	Importa el modelo de red neuronal para regresión.
<code>hidden_layer_sizes=(100,)</code>	Define una arquitectura con una capa oculta de 100 neuronas. La coma indica que es una tupla de un elemento.
<code>activation='relu'</code>	Usa la función de activación ReLU en las capas ocultas. Esta función permite modelar no linealidad.
<code>solver='adam'</code>	Utiliza el optimizador Adam, ampliamente usado por su estabilidad y eficiencia.
<code>max_iter=500</code>	Define un máximo de 500 iteraciones de entrenamiento.
<code>random_state=42</code>	Fija la semilla aleatoria para obtener resultados reproducibles.
<code>early_stopping=True</code>	Detiene el entrenamiento si el rendimiento de validación deja de mejorar. Esto ayuda a prevenir sobreajuste.
<code>mlp_model.fit(X_train_scaled, y_train)</code>	Entrena la red neuronal. El escalamiento es especialmente importante para este tipo de modelo.
<code>y_pred_mlp = mlp_model.predict(X_test_scaled)</code>	Genera predicciones sobre los datos de prueba escalados.
<code>mse_mlp = mean_squared_error(y_test, y_pred_mlp)</code>	Calcula el MSE de la red neuronal.
<code>r2_mlp = r2_score(y_test, y_pred_mlp)</code>	Calcula el R^2 de la red neuronal.

5.2. Resultados obtenidos

Resultado	Valor
Primeras 5 predicciones	[0.3747, 1.1748, 4.7541, 2.5477, 2.9169]
MSE	0.3017
R^2	0.7697

5.2.1 Interpretación

La red neuronal obtiene el menor MSE y el mayor R^2 entre los tres modelos evaluados. Un R^2 de 0.7697 indica que el modelo explica aproximadamente el 76.97% de la variabilidad del valor mediano de las viviendas en el conjunto de prueba. Esto representa una mejora relevante respecto de la regresión lineal y del árbol de decisión con profundidad limitada.

No obstante, este mejor desempeño debe analizarse junto con su complejidad. Una red neuronal puede capturar patrones complejos, pero es menos interpretable. Además, requiere una preparación cuidadosa de los datos, control de hiperparámetros y monitoreo del sobreajuste.

6. Comparación e interpretación integrada de modelos

6.1. Código de comparación

```
import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

# Crear un DataFrame para comparar los resultados
results = pd.DataFrame({
    'Modelo': ['Regresión Lineal', 'Árbol de Decisión', 'Red
Neuronal (MLP)'],
    'MSE': [mse_linear, mse_tree, mse_mlp],
    'R2': [r2_linear, r2_tree, r2_mlp]
})

display(results.sort_values(by='R2', ascending=False))

# Visualización de los resultados
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Gráfico para MSE

sns.barplot(ax=axes[0], x='Modelo', y='MSE', data=results,
palette='viridis')

axes[0].set_title('Comparación de MSE entre Modelos')
axes[0].set_ylabel('Error Cuadrático Medio (MSE)')
axes[0].set_xticklabels(axes[0].get_xticklabels(), rotation=45,
ha='right')

# Gráfico para R2

sns.barplot(ax=axes[1], x='Modelo', y='R2', data=results,
palette='magma')

axes[1].set_title('Comparación de R^2 entre Modelos')
axes[1].set_ylabel('Coeficiente de Determinación (R^2)')
axes[1].set_ylim(0, 1)

axes[1].set_xticklabels(axes[1].get_xticklabels(), rotation=45,
ha='right')

plt.tight_layout()

plt.show()
```

Línea de código	Explicación
<code>results = pd.DataFrame({...})</code>	Construye una tabla con los nombres de los modelos y sus métricas de evaluación.
<code>display(results.sort_values(by='R2', ascending=False))</code>	Ordena los modelos desde el mayor al menor R^2 . Esto facilita identificar el modelo con mayor poder explicativo.
<code>fig, axes = plt.subplots(1, 2, figsize=(14, 6))</code>	Crea una figura con dos gráficos en una fila: uno para MSE y otro para R^2 .
<code>sns.barplot(... MSE ...)</code>	Genera un gráfico de barras para comparar el error cuadrático medio. En esta métrica, valores menores son mejores.
<code>axes[0].set_title(...)</code> y <code>axes[0].set_ylabel(...)</code>	Define título y etiqueta del eje vertical del gráfico de MSE.
<code>axes[0].set_xticklabels(...)</code>	Rota las etiquetas del eje X para mejorar la legibilidad.
<code>sns.barplot(... R2 ...)</code>	Genera un gráfico de barras para comparar R^2 . En esta métrica, valores mayores son mejores.
<code>axes[1].set_ylim(0, 1)</code>	Fija el eje vertical entre 0 y 1 porque R^2 suele interpretarse en ese rango cuando el modelo tiene desempeño razonable.
<code>plt.tight_layout()</code>	Ajusta los espacios para evitar superposición de etiquetas.
<code>plt.show()</code>	Muestra los gráficos en el notebook.

6.2. Tabla comparativa de resultados

Modelo	MSE	R^2	Lectura del resultado
Red Neuronal (MLP)	0.301732	0.769742	Presenta el menor error y el mayor poder explicativo. Es el mejor desempeño predictivo del ejercicio.
Árbol de Decisión	0.524515	0.599732	Mejora levemente la regresión lineal y captura relaciones no lineales, aunque con profundidad limitada.
Regresión Lineal	0.555892	0.575788	Funciona como línea base interpretable, pero tiene menor capacidad para capturar relaciones complejas.

6.3. Visualización de resultados

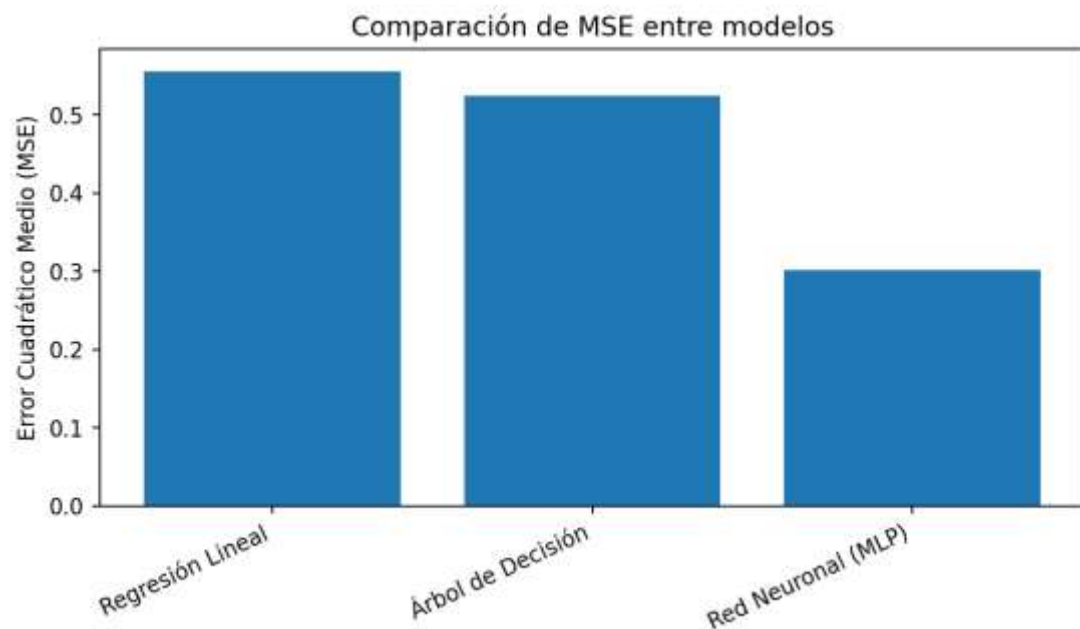


Figura 1. Comparación del MSE entre modelos. En MSE, valores menores indican mejor desempeño predictivo.

Fuente: Elaboración propia.

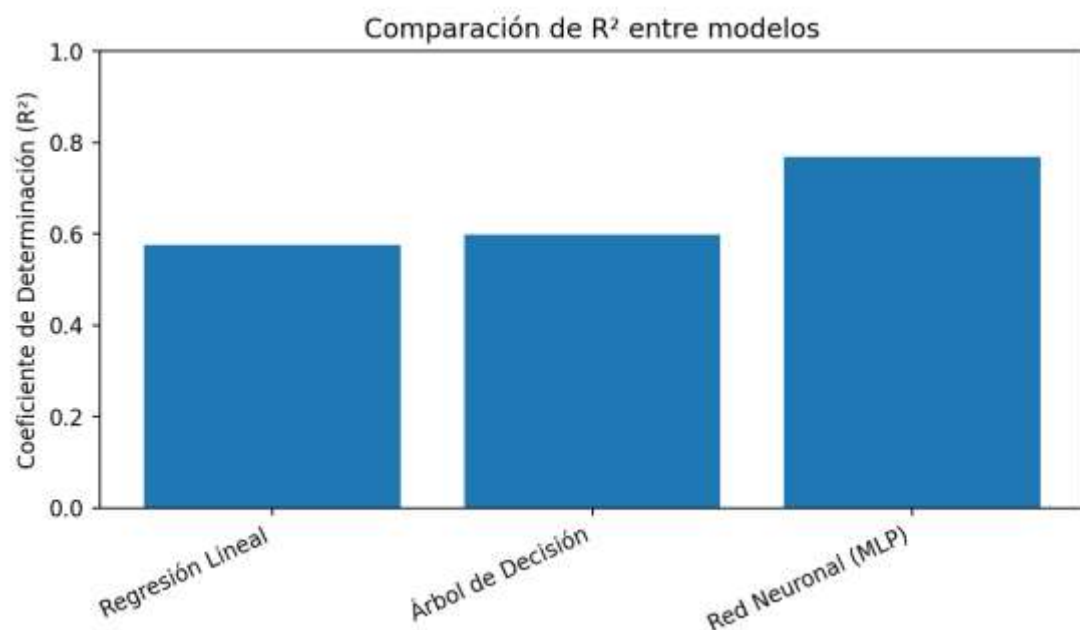


Figura 2. Comparación del R^2 entre modelos. En R^2 , valores más cercanos a 1 indican mayor proporción de varianza explicada.

Fuente: Elaboración propia.

6.3.1. Interpretación integrada

La comparación muestra que la red neuronal MLP es el mejor modelo en términos de desempeño cuantitativo, porque obtiene simultáneamente el menor MSE y el mayor R^2 (ver Figura 1 y 2). Esto indica que sus predicciones se alejan menos de los valores reales y que explica una mayor proporción de la variabilidad del precio mediano de las viviendas.

El árbol de decisión se ubica en segundo lugar. Su desempeño es mejor que la regresión lineal, lo que sugiere que existe cierta no linealidad en los datos. Sin embargo, al estar limitado a una profundidad máxima de 5, su capacidad de ajuste queda restringida. Esta restricción es útil para evitar sobreajuste, pero puede impedir capturar estructuras más complejas.

La regresión lineal queda como modelo base. Su valor pedagógico es importante porque permite introducir el flujo de entrenamiento, predicción y evaluación de manera clara. Sin embargo, su menor R^2 indica que el supuesto de linealidad no representa completamente la estructura del problema.

Recursos pedagógicos de la unidad

Preguntas de reflexión

- ¿Por qué es necesario separar variables de entrada y variable objetivo antes de entrenar un modelo supervisado?
- ¿Qué problema metodológico se produce si el escalador se ajusta con todo el dataset antes de dividir entrenamiento y prueba?
- ¿Por qué una red neuronal puede obtener mejor desempeño que una regresión lineal en un problema de precios de vivienda?
- ¿En qué situación preferiría un modelo lineal aunque tenga menor R^2 que una red neuronal?
- ¿Cómo se relacionan MSE y R^2 con la toma de decisiones sobre el mejor modelo?

Preguntas de autoevaluación

- Identifique la variable objetivo del dataset California Housing.
- Explique qué significa `test_size=0.20` en `train_test_split`.
- Indique por qué `random_state=42` permite reproducibilidad.
- Explique qué calcula `StandardScaler`.
- Compare MSE y R^2 indicando cuándo un valor es mejor.
- Señale qué modelo obtuvo mejor resultado en el ejercicio y justifique con métricas.
- Explique una ventaja y una limitación de la regresión lineal.
- Explique una ventaja y una limitación de los árboles de decisión.
- Explique una ventaja y una limitación de las redes neuronales MLP.

Glosario de conceptos fundamentales

Concepto	Definición
Aprendizaje supervisado	Tipo de aprendizaje en el que el modelo se entrena con ejemplos que incluyen una variable objetivo conocida.
Regresión	Problema supervisado cuyo objetivo es predecir un valor numérico continuo.
Variable objetivo	Variable que se desea predecir. En este caso, <code>MedHouseVal</code> .
Características	Variables de entrada utilizadas por el modelo para generar predicciones.
Conjunto de entrenamiento	Subconjunto de datos usado para ajustar los parámetros del modelo.
Conjunto de prueba	Subconjunto reservado para evaluar el desempeño con datos no vistos.
Escalamiento	Transformación que coloca las variables en escalas comparables.
MSE	Error Cuadrático Medio. Promedio de los errores al cuadrado entre valores reales y predichos.

R^2	Coeficiente de determinación. Proporción de varianza explicada por el modelo.
Sobreajuste	Situación en que el modelo aprende demasiado bien los datos de entrenamiento y generaliza mal.
Early stopping	Técnica que detiene el entrenamiento cuando el desempeño de validación deja de mejorar.

Síntesis final



En este apunte se desarrolló un flujo completo de regresión supervisada en Python utilizando el dataset California Housing. Primero se cargaron los datos y se verificó la ausencia de valores nulos. Luego se separaron las variables de entrada y la variable objetivo, se realizó una división entrenamiento-prueba y se aplicó escalamiento estándar a las características.

Posteriormente se implementaron tres modelos: regresión lineal, árbol de decisión y red neuronal MLP. Cada modelo fue entrenado con los datos de entrenamiento y evaluado en el conjunto de prueba mediante MSE y R^2 . La regresión lineal permitió establecer una línea base interpretable; el árbol de decisión mostró una mejora moderada al capturar relaciones no lineales; y la red neuronal obtuvo el mejor desempeño global, con MSE de 0.3017 y R^2 de 0.7697.

El ejercicio permite comprender que seleccionar un modelo no depende únicamente de obtener una métrica alta. También se debe considerar interpretabilidad, complejidad, riesgo de sobreajuste, costo computacional y pertinencia del modelo frente al problema analítico.